

מעבדת ארכיטקטורת מעבדים

מתקדמת ומאיצי חומרה

36114693

דוח מכין מעבדה 3

אדר שפירא 209580208

יהונתן דדכה 211468582

מבוא

בפרויקט זה תכננו, מימשנו וביצענו וריפיקציה למעבד Simple RISC Multi-Cycle CPU. המטרה המרכזית הייתה לבנות מערכת שלמה הכוללת חומרה ותוכנה מבוססת בקר, תוך הפרדה מתודולוגית ברורה בין יחידת הבקרה לבין אפיק הנתונים. סביבת הבדיקה שלנו תוכננה לבצע סימולציה מבוססת קבצים. המערכת טוענת קוד מכונה לזיכרון ההוראות מקובץ ITCMinit.txt, מאתחלת את זיכרון הנתונים מקובץ DTCMinit.txt, ושומרת את תוצאות הריצה של המעבד לתוך קובץ DTCMcontent.txt.

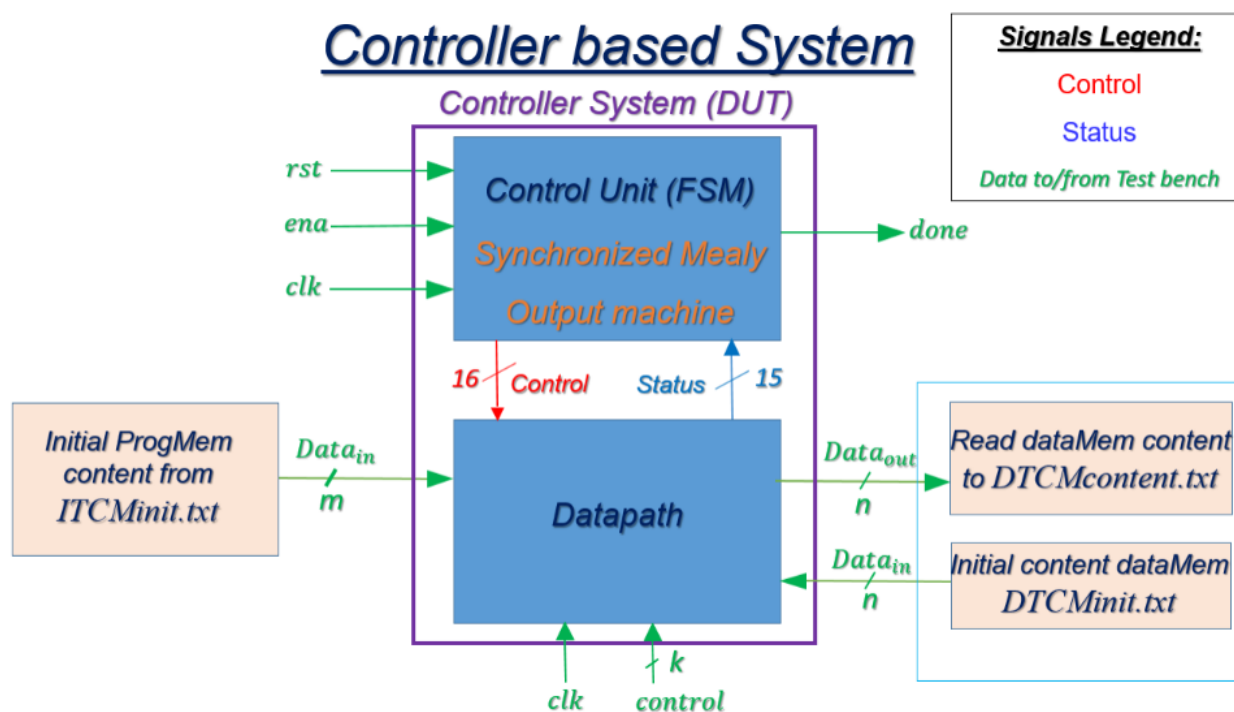


Figure 1: Overall DUT structure

ארכיטקטורת המערכת

המערכת שבנינו מורכבת ממספר בלוקים הפועלים בסנכרון:

המערכת השלמה: המבנה מפריד בין מכונת המצבים (FSM) לבין ה-Datapath, כאשר אותות בקרה מנווטים את המידע ואותות סטטוס חוזרים חזרה לבקר. המערכת מתממשקת עם ה-Testbench ששולט על אתחול וקריאת הזיכרונות.

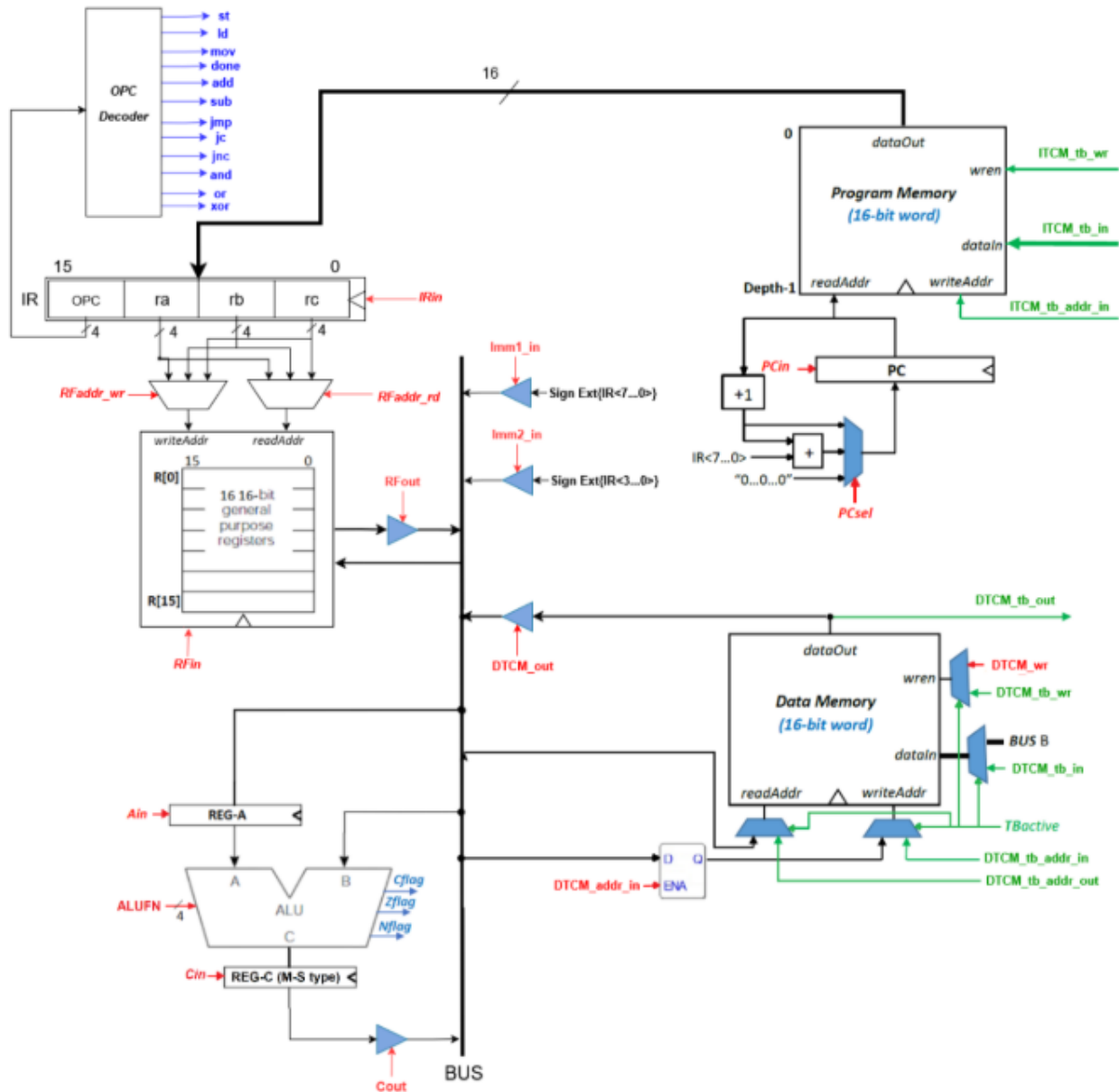
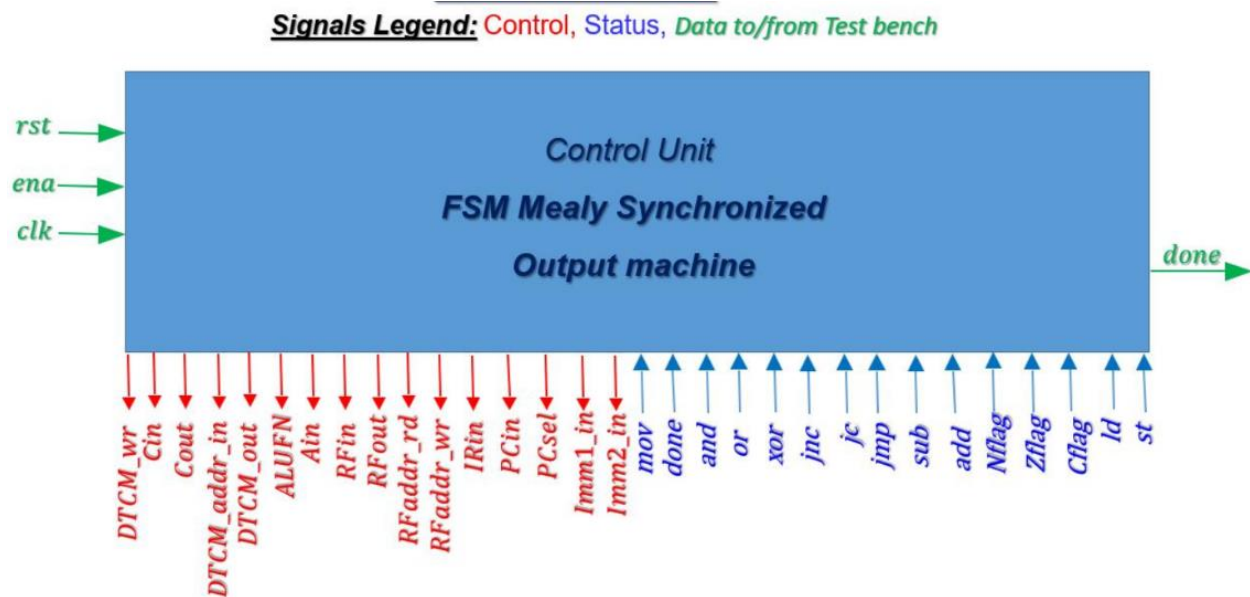


Figure 2: SRMC Datapath structure of a 1-BUS multicycle CPU

יחידת הנתונים (Datapath): ה-Datapath תוכנן בארכיטקטורת אפיק יחיד (1-BUS multicycle CPU). הרכיב מכיל את מונה התוכנית (PC), רגיסטר הפקודה (IR), מבנה אוגרים (Register File), ויחידת חישוב (ALU) המפיקה דגלי סטטוס.

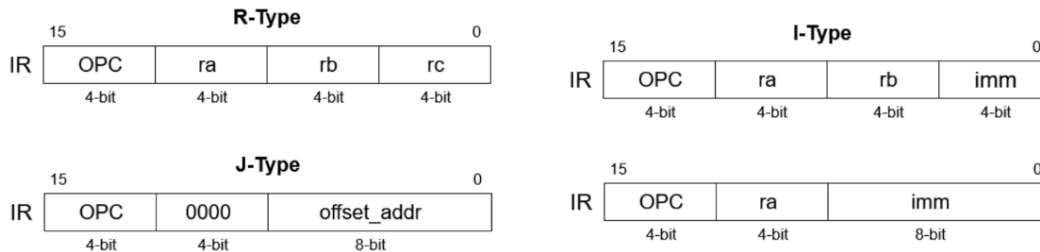
יחידת הבקרה (Control): ממומשת כמכונת מצבים מסונכרנת (FSM Mealy Synchronized Output machine). המונה מקבלת את ה-Opcode, ומפיקה את כל אותות הבקרה ל-Datapath, ובסיום מוציאה את האות done.



סט הפקודות והזיכרונות

הגדרנו ארכיטקטורת תוכנה וזיכרון ייעודית ברוחב 16-ביט :

פורמט ההוראות: הפקודות מחולקות ל-3 סוגים: I-Type, R-Type, J-Type.



מצבי מיעון ופעולות: המעבד תומך ב-5 מצבי מיעון: Indexed, Register, Immediate, Direct, Indirect. מוגדרות 16 פקודות Opcode (כולל add, sub, jmp, mov, ld, st, done), כאשר האוגר R[0] מחווט תמיד לערך אפס.

ADDRESS MODE	SYNTAX	INSTRUCTION FORMAT	EXAMPLE	OPERATION
Register	add ra,rb,rc	R-Type	add r4,r2,r1	$R[4] \leftarrow R[2] + R[1]$
Immediate	mov ra,imm	I-Type	mov r5,0x30 mov r5,Var	$R[5] \leftarrow 0x30$ $R[5] \leftarrow \text{Var}$
Direct	ld ra,imm(r0)		ld r4,0x20(r0) ld r4, Var(r0)	$R[4] \leftarrow M[0x20]$ $R[4] \leftarrow M[\text{Var}]$
Indirect	ld ra,0(rb)		ld r4,0(r3)	$R[4] \leftarrow M[R[3]]$
Indexed	ld ra,imm(rb)		ld r4,0x20(r3) ld r4, Var(r3)	$R[4] \leftarrow M[0x20 + R[3]]$ $R[4] \leftarrow M[\text{Var} + R[3]]$

Instruction Format	Decimal value	OPC	Instruction	Explanation	N	Z	C
R-Type	0	0000	add ra,rb,rc nop	$R[ra] \leftarrow R[rb] + R[rc]$ $R[0] \leftarrow R[0] + R[0]$ (<i>emulated instruction</i>)	*	*	*
	1	0001	sub ra,rb,rc	$R[ra] \leftarrow R[rb] - R[rc]$	*	*	*
	2	0010	and ra,rb,rc	$R[ra] \leftarrow R[rb] \text{ and } R[rc]$	*	*	*
	3	0011	or ra,rb,rc	$R[ra] \leftarrow R[rb] \text{ or } R[rc]$	*	*	*
	4	0100	xor ra,rb,rc	$R[ra] \leftarrow R[rb] \text{ xor } R[rc]$	*	*	*
	5	0101	unused				
	6	0110	unused				
J-Type	7	0111	jmp offset_addr	$PC \leftarrow PC + 1 + \text{offset_addr}$	-	-	-
	8	1000	jc /jhs offset_addr	If (Cflag==1) $PC \leftarrow PC + 1 + \text{offset_addr}$	-	-	-
	9	1001	jnc /jlo offset_addr	If (Cflag==0) $PC \leftarrow PC + 1 + \text{offset_addr}$	-	-	-
	10	1010	unused				
	11	1011	unused				
I-Type	12	1100	mov ra,imm	$R[ra] \leftarrow \text{imm}$	-	-	-
	13	1101	ld ra,imm(rb)	$R[ra] \leftarrow M[\text{imm} + R[rb]]$	-	-	-
	14	1110	st ra,imm(rb)	$M[\text{imm} + R[rb]] \leftarrow R[ra]$	-	-	-
Special	15	1111	done	Signals the TB to read the DTCM content	-	-	-

מבנה הזיכרונות: קיימת הפרדה חשובה במימוש הזיכרונות:

זיכרון התוכנית (Program Memory/ITCM) ומבנה האוגרים (Register File) ממומשים כ-Single Port RAM ללא אוגר ביציאה (unregistered output).

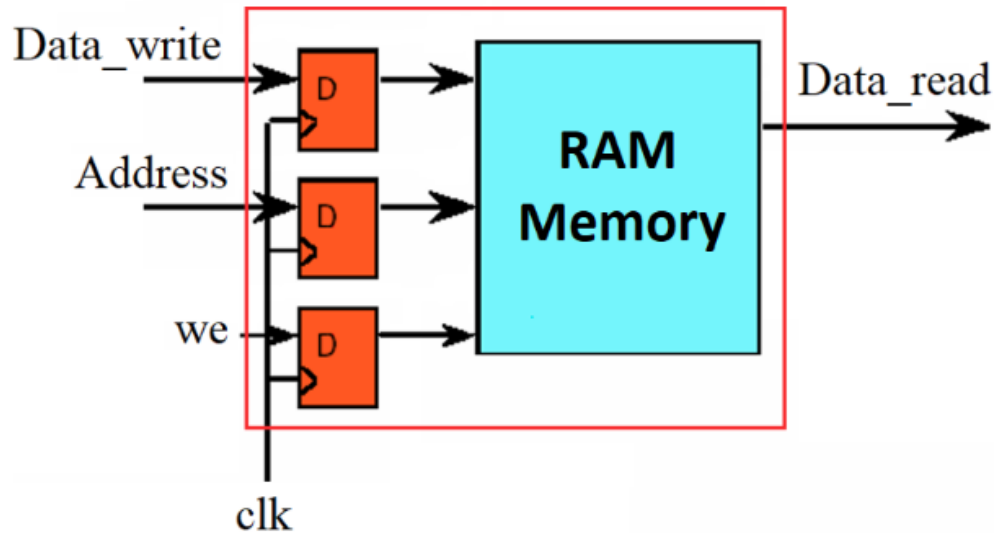


Figure 5: Single Port RAM with an unregistered output. This structure is used for Register File and Program Memory (ITCM) with a given VHDL code.

זיכרון הנתונים (Data Memory/DTCM) ממומש כ-Single Port RAM עם אוגר ביציאה (registered output).

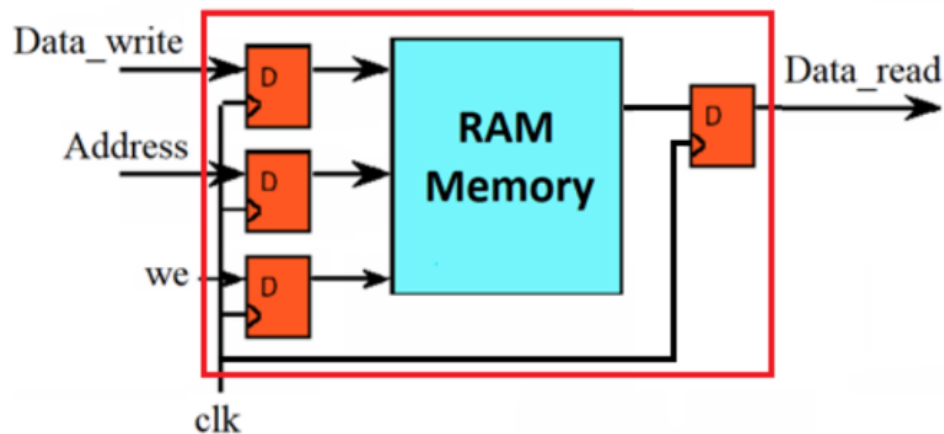


Figure 6: Single Port RAM with a registered output. This structure is used for Data Memory (DTCM) with a given VHDL code.

תוצאות סימולציה

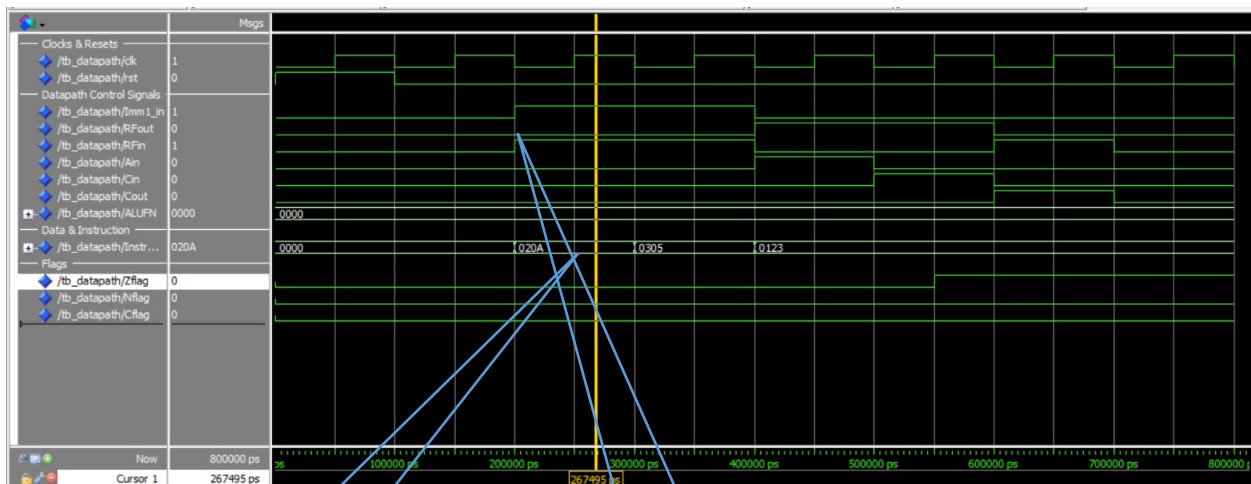
במהלך הסימולציות, יצרנו סביבת בדיקה מקיפה הבוחנת את המערכת תחת תרחישים ומקרי קיצון שונים. התמקדנו בנקודות זמן קריטיות כדי להוכיח את תקינות המערכת.

תוצאות סימולציה של רכיב Datapath

מקרה בדיקה 1: טעינת נתונים התחלתית

האות Instruction_in מקבל את הערכים 020A ואז 0305. במקביל, אותות הבקרה Imm1_in ו-RFin נמצאים ב-'1' לוגי.

בשלב זה אנו בודקים את יכולת ה-Datapath לטעון ערכים מיידיים (Immediate) ישירות לתוך ה-Register File. ניתן לראות כיצד מנותב המידע (המספרים 10 ו-5) אל האוגרים R2 ו-R3 ללא מעבר דרך ה-ALU, באמצעות הדלקת האותות Imm1_in (ניתוב ה-Immediate לאפיק) ו-RFin (כתיבה לאוגר).



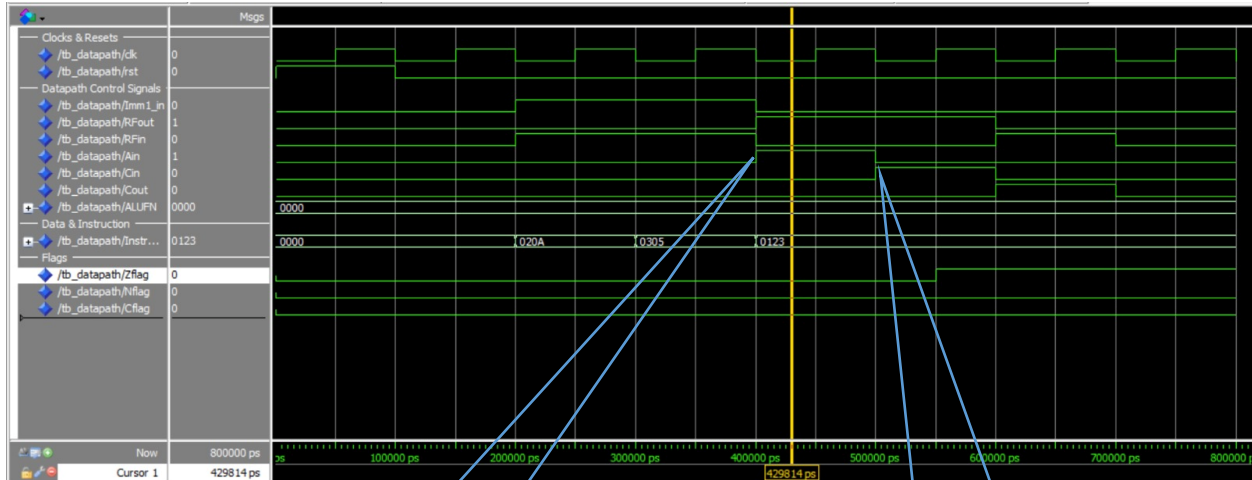
פקודות טעינה Immediate

ניתוב וכתיבה ל-Register File

מקרה בדיקה 2: קריאת אופרנדים וחישוב ALU

האות Instruction_in משתנה ל-0123 (שזה בעצם ADD R1, R2, R3).

ביצוע פקודת ה-ADD. במחזור השעון הראשון, ה-Datapath קורא את האופרנד הראשון (R2) ושומר אותו באוגר המבוא A (האות Ain עולה). במחזור השעון העוקב (ns500), אנו קוראים את האופרנד השני (R3), מעבירים את ה-Opcode של החיבור ל-ALU (ALUFN=0000), ודוגמים את התוצאה לתוך אוגר היציאה C (האות Cin עולה).

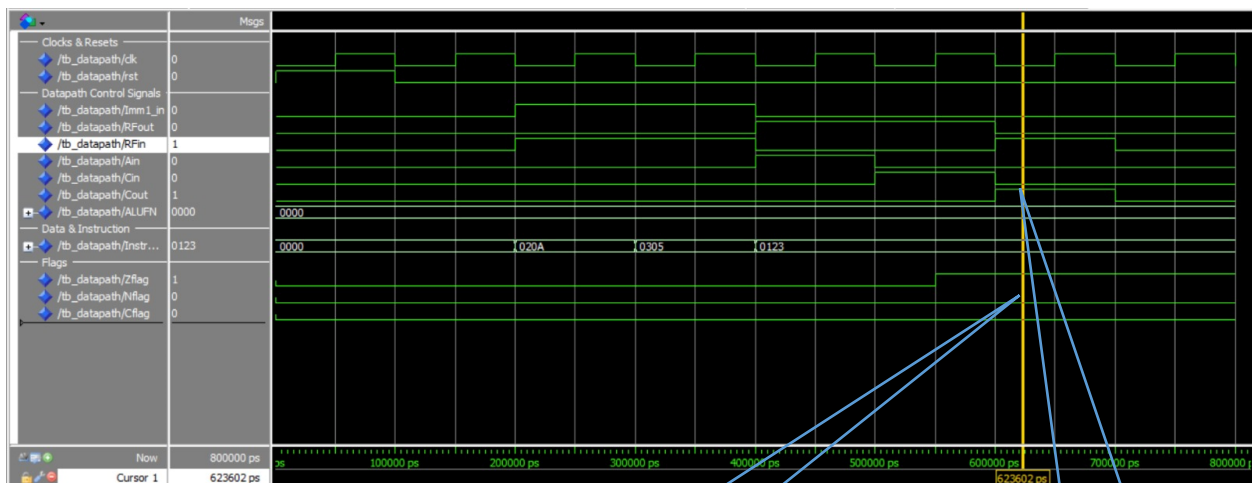


אופרנד ראשון מR2 מוזרם לאפיק
באמצעות RfOut ונדגם סינכרונית
לתוך אוגר המבוא של ALU עם עליית
Ain האות

אופרנד מאוגר R3 מוזרם לאפיק ה-
ALU מבצע חיבור (ALUFN=0000)
והאות Cin עולה כדי לדגום את
התוצאה לרגיסטר REG-C

מקרה בדיקה 3: שלב ה-Write-Back

סיום הפעולה. התוצאה שנשמרה באוגר C נכתבת חזרה לאפיק הנתונים (על ידי הדלקת Cout), באותו מחזור שעון נשמרת אל תוך ה-Register File לאוגר היעד R1 (על ידי הדלקת RFin).



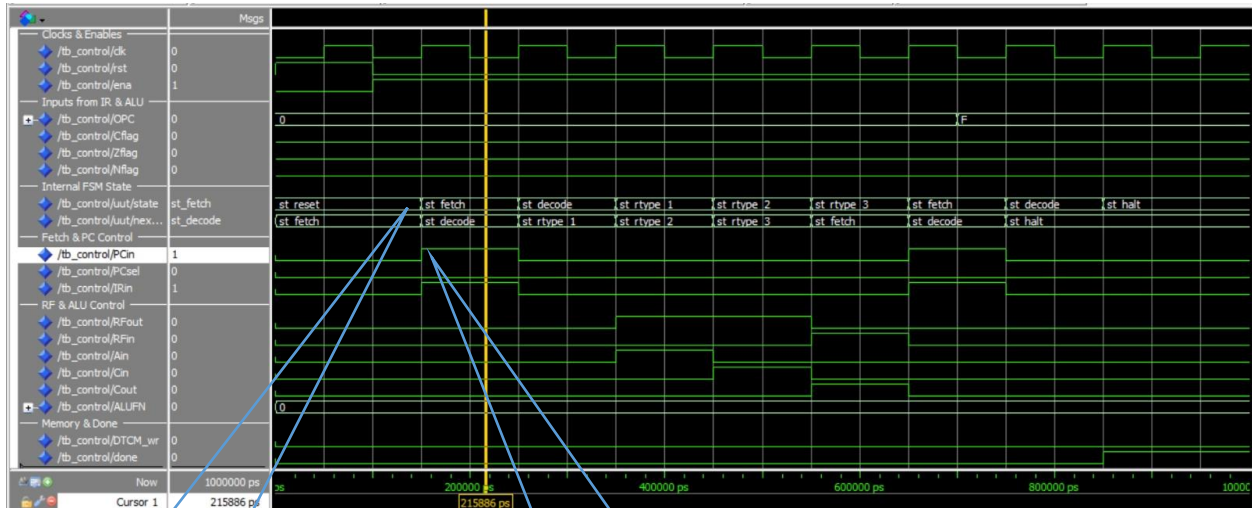
שלושת האותות נשארים על 0 לוגי
מכיוון שפעולת החיבור הניבה תוצאה
חיובית שאינה 0.

הערך המחושב מאוגר reg-c מוזרם
באמצעות האות Cout ונכתב סינכרונית
לאוגר היעד R1 עם עליית אות הבקרה
RFin

תוצאות סימולציה של רכיב Control

מקרה בדיקה 1: שלב ה-Fetch & Decode

בתמונה ניתן לראות את מכונת המצבים יוצאת ממצב `st_reset` ועוברת למצב `st_fetch`. במצב זה, ה-FSM מדליק את האותות `PCin` ו-`IRin` כדי לקדם את מונה התוכנית ולדגום את הפקודה החדשה מזיכרון ההוראות. מיד לאחר מכן, המכונה עוברת למצב `st_decode` לפענוח ה-`Opcode` (אשר מקבל כאן את הערך 0 עבור פקודת יחידת ה-`ALU`).



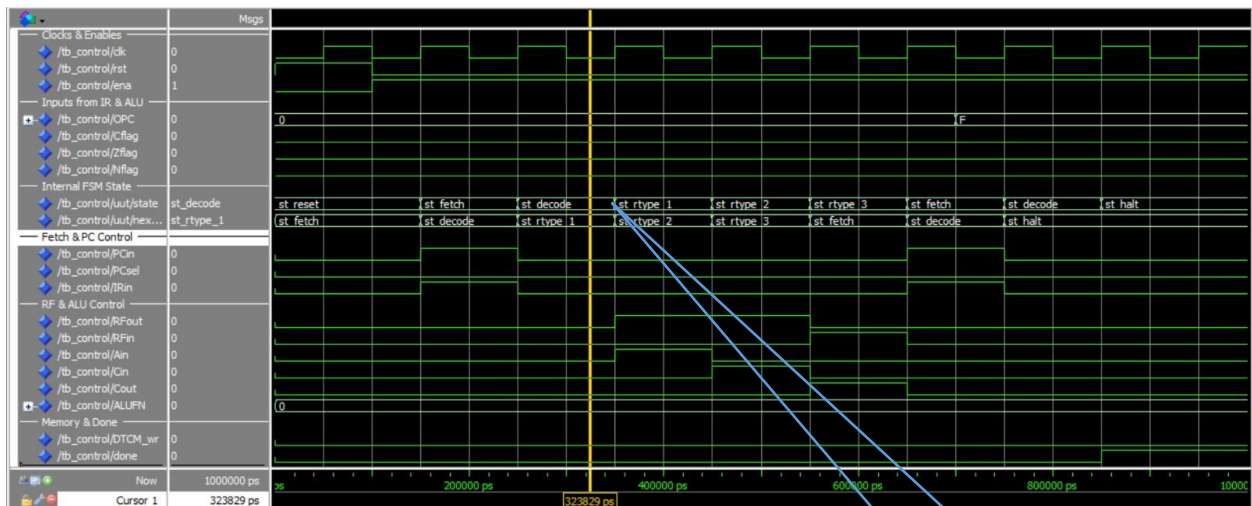
מכונת המצבים יוצאת ממצב `st_reset` ועוברת סנכרונית למצב `st_fetch` לקראת שליפת הפקודה הראשונה מהזכרון

אותות הבקרה `PCin`, `IRin` עולים יחד ל 1 לוגי כדי לקדם את מונה התוכנית ולדגום את הפקודה החדשה לתוך ה-`IR`.

מקרה בדיקה 2: ביצוע פקודת ADD

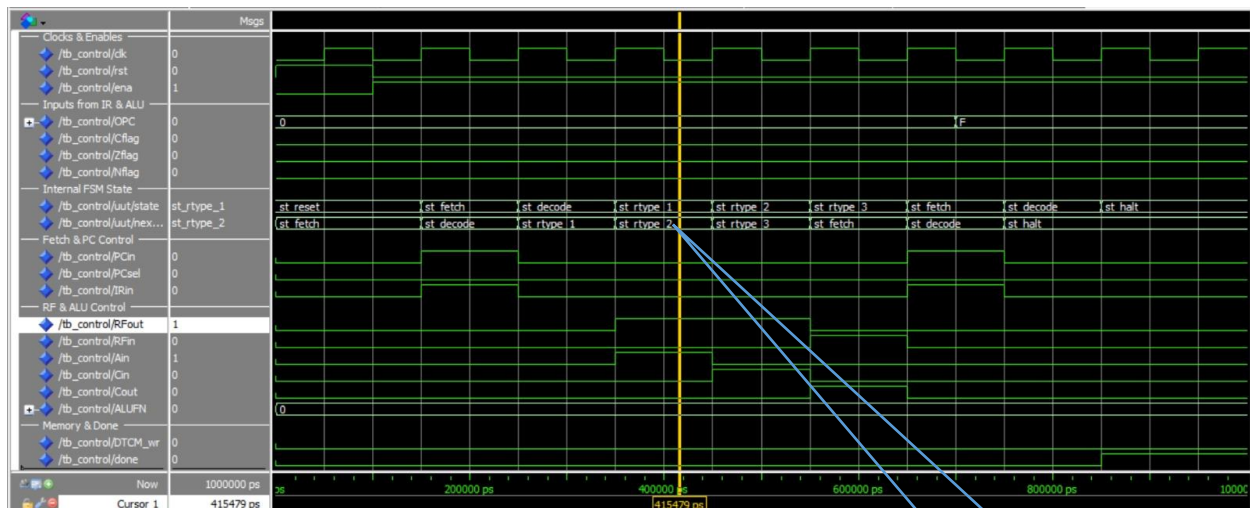
הוכחה לפעולת הפיצול (Branching) של ה-FSM. מכיוון שזווה `OPC=0`, המכונה עוברת לשרשרת המצבים הרלוונטית (`R-Type`). נראה כאן סנכרון מושלם ל-Datapath:

במצב `st_rtype_1` מופעל `Ain` לקריאת אופרנד ראשון.



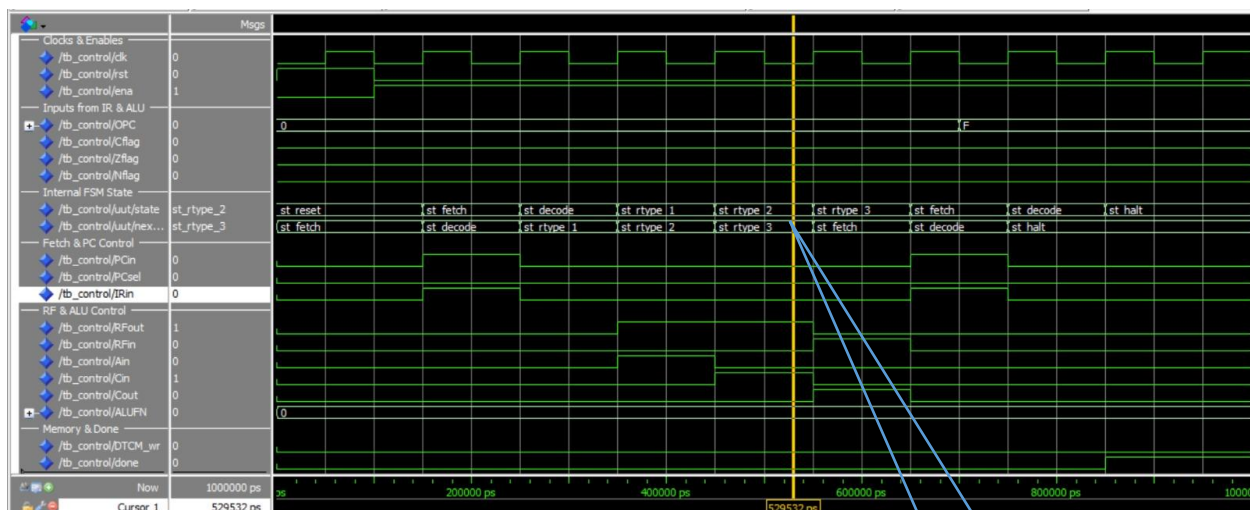
מכונת המצבים מזהה `OPC=0` ועוברת למצב `st_rtype=1` המפעיל את האותות `Ain` ו-`RFout` כדי לקרוא את האופרנד הראשון לאוגר `REG-A`

במצב st_rtype_2 מופעל Cin יחד עם ALUFN לביצוע החיבור.



המערכת עוברת למצב sr_rtype=2 שבו האות Cin עולה ל1 לוגי כדי לדגום את תוצאת החישוב של ALU אל תוך REG-C

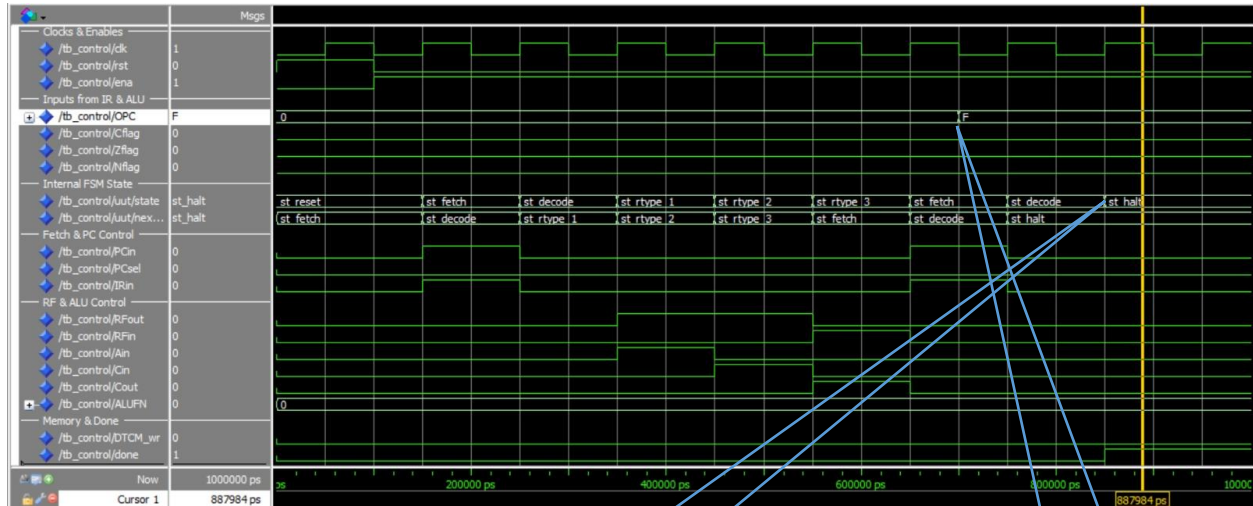
במצב st_rtype_3 מופעלים Cout ו-RFin לכתובת התוצאה חזרה ל-Register File.



המערכת מגיעה למצב sr_rtype=3 המפעיל את האותות Cout ו-RFin כדי לכתוב את התוצאה הסופית מ-REG-C אל register file

מקרה בדיקה 3: זיהוי פקודת עצירה ומצב Halt

לאחר סיום הפקודה הראשונה, ה-FSM חוזר למצב `st_fetch`. כעת, ה-Opcode משתנה ל-F (פקודת DONE). לאחר הפענוח, מכונת המצבים מנווטת מיד למצב סופי `st_halt`. במצב זה, כל אותות הבקרה כבויים, והאות `done` עולה ל-'1' לוגי קבוע כדי לעצור את ריצת המערכת.



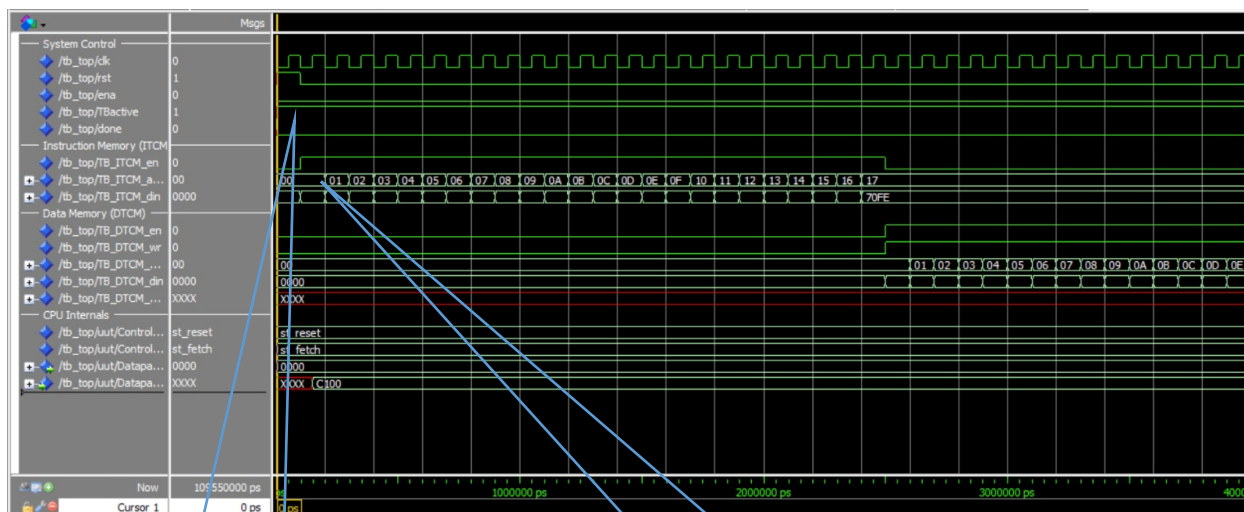
מכונת המצבים מפענת את opcode
ועוברת למצב הסופי st_halt
את אות המוצא done ל 1 ועוצר את
המערכת.

קלט opcoden משתנה לערך F
(פקודת DONE), המאותת ליחידת
הבקרה שהגענו לפקודת סיום
התוכנית.

תוצאות סימולציה של כל המערכת

מקרה בדיקה 1: שלב האתחול - טעינת הזיכרון (Memory Initialization)

שלב אתחול המערכת (Step 2 & 3 ב-Testbench). ניתן לראות כי המעבד כבוי (ena=0) וה-Testbench שולט באפיקי הזיכרון (TBActive=1). בשלב זה קבצי הקידוד נטענים ישירות לתוך זיכרון ההוראות (ITCM) וזיכרון הנתונים (DTCM). הכתובות (TB_DTCM_addr) עולות ברצף, והנתונים נכתבים פנימה.

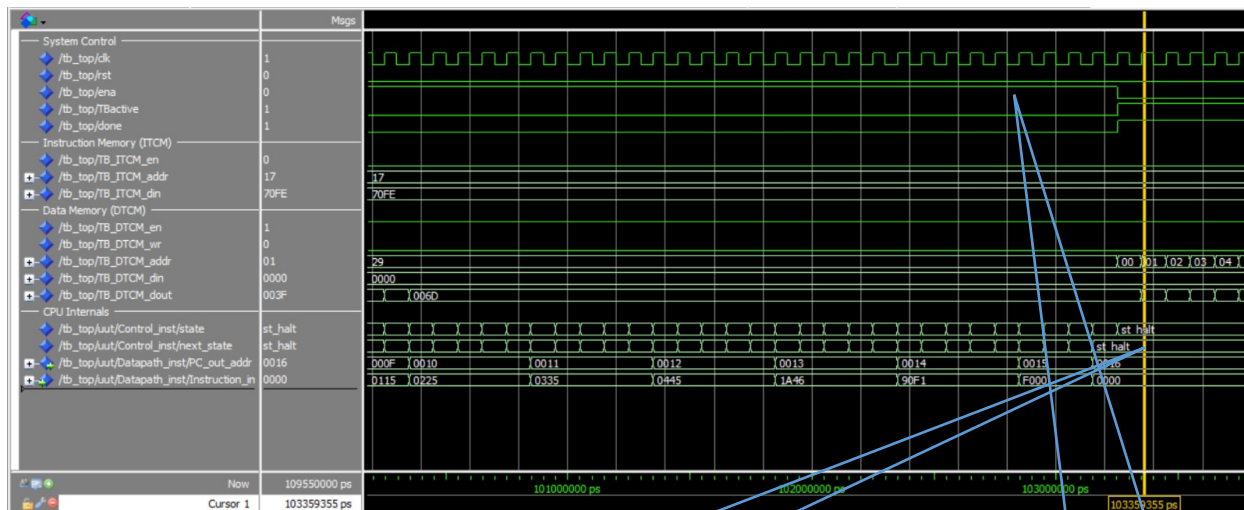


האות TBActive 1 והאות ena 0 מעבירות את השליטה על הזכרונות מהמעבד אל testbench

ה-Testbench מזרים פקודות ונתונים לתוך הזיכרונות, כאשר הכתובות מתקדמות סדרתית בכל מחזור לצורך טעינה מלאה

מקרה בדיקה 2: העברת השליטה למעבד ותחילת ריצה (Execution Start)

תחילת פעולת המעבד. ה-Testbench מסיים לטעון את הזיכרונות ומעביר את השליטה למעבד על ידי הורדת TBActive והדלקת ena. מיד לאחר מכן, מכונת המצבים יוצאת ממצב Reset ומתחילה במחזור ה-Fetch הראשון. ניתן לראות את מונה התוכנית (PC) מתקדם ואת ההוראות נשלפות מזיכרון ה-ITCM (Instruction_in).

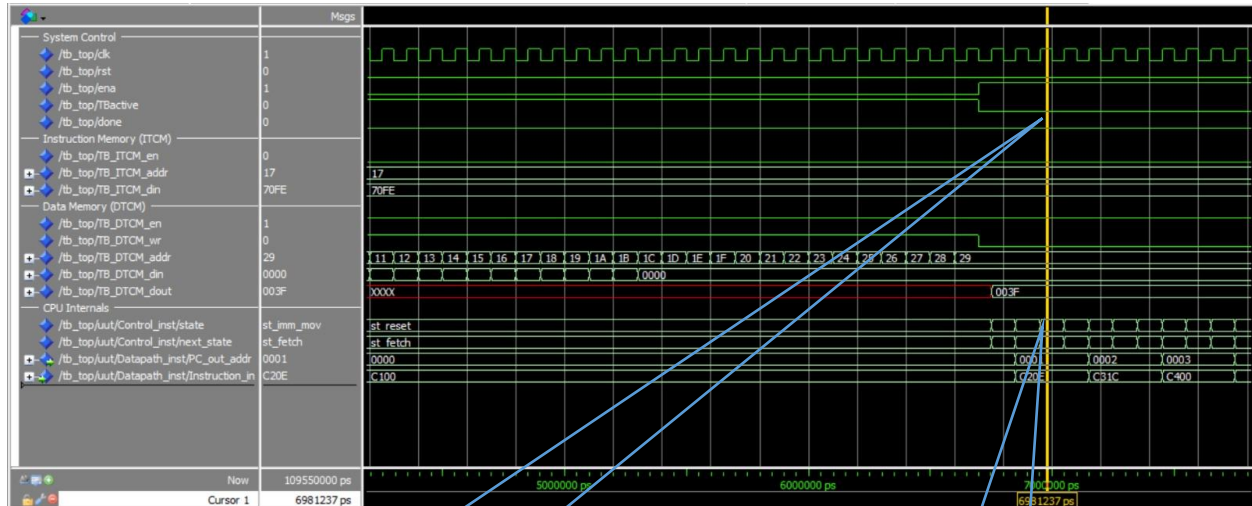


מונה התוכנית מתחיל לעלות סדרתית, ויחידת הבקרה יוצאת ממצב reset אל מחזור ה fetch הראשון

העלאת האות TBActive ל1 והורדת האות ena ל0 מעבירות את השליטה על הזכרונות מהמעבד אל testbench

מקרה בדיקה 3: סיום התוכנית בצורה תקנית (Graceful Exit)

סיום מוצלח של ריצת האפליקציה. המעבד מסיים את הלולאות, שולף מזיכרון ה-ITCM את פקודת העצירה (F000 ב-Hex), ועובר למצב `st_halt`. בשלב זה האות `done` עולה ל-'1', מה שמאותת ל-Testbench שהחישוב הסתיים. מיד לאחר מכן, ה-Testbench לוקח בחזרה את השליטה (`TBactive=1`) כדי לקרוא את תוצאות החישוב מתוך ה-DTCM אל קובץ הפלט.



עם הכניסה למצב העצירה, st_halt יחדית הבקרה מרימה את אות המוצא done ל-'1' לוגי כדי לאות ל Testbench-שהתוכנית הסתיימה.

המעבד מפענח את פקודת הסיום F000 הניזונה מזיכרון ה-ITCM וגורם ליחידת הבקרה להעביר את המערכת למצב st_halt לעצירה מלאה.